

Lecture #6

Instructor: Yunseok Choi
(ys.choi@skku.edu)

Outlier Detection

Outlier Detection

- Outlier Detection(이상치 탐지)은 데이터 내에서 정상적인 패턴과는 다른, 드물고 예상치 못한 패턴(outlier, anomaly, abnormal)을 찾아내는 기법이다.
- 대량의 정상 데이터 속에서 **소수의 비정상 데이터를** 효과적으로 구별하기 위해 사용된다.

Outlier Detection의 중요성

- 정상 데이터가 대부분을 차지하는 비대칭적 구조에서 이상치를 찾아야 하는 경우가 많다.
- 이상치는 시스템 오류, 사기 행위, 질병의 조짐 등 다양한 문제의 **조기 신호**일 수 있으며, 때로는 치명적인 손실로 이어질 수 있다.
- 이상치는 사전에 정의되지 않은 경우가 많아, **전통적인 rule-based 시스템으로는 탐지가 불가능**하다.

Outlier Detection 활용 분야 1

- **Fraud Detection (이상 거래 탐지)**: 신용, 보험 등 신뢰성이 중요한 금융계 분야에서 이상치를 탐지하여 부정 사용 및 악용 방지
- **Intrusion Detection System (침입 탐지 시스템)**: 네트워크 및 시스템 접근 로그에서 이상치를 탐지하여 해킹이나 무단 접근 등의 보안 사고 예방

Outlier Detection 활용 분야 2

- **Predictive maintenance (설비 예지 보전)**: 설비 센서 데이터에서 비정상 신호는 오류나 장비 고장을 의미할 수 있기 때문에 조기에 이상을 발견해 유지보수 비용 절감 가능
- **Customer Churn Prediction (고객 이탈 예측)**: 고객 행동 패턴에서 갑작스러운 사용 감소 및 부정적 반응을 탐지해 잠재 이탈 고객을 조기에 파악하고 유지 전략 마련

Outlier Detection 활용 분야 3

- **Healthcare Monitoring (환자 생체 신호 이상 탐지)**: 환자의 심박수, 산소포화도, 체온 등 생체 데이터를 실시간으로 모니터링하여 이상치를 조기에 탐지해 응급 대응
- **Smart City / IoT 기반 이상 탐지**: 교통 흐름, 공기질, 에너지 사용량 등의 센서 데이터에서 이상 패턴을 탐지하여 도시 자원 낭비나 긴급 상황 (예. 화재, 정전 등)을 예방

Outlier Detection 접근 방식 1

1. Supervised Outlier Detection

- 정상/비정상 라벨이 포함된 데이터로 학습
- 일반적인 분류 (classification) 문제로 처리 가능
- 현실에서는 이상치 수가 적어 **클래스 불균형 문제** 존재
- Accuracy는 높는데 실제 비정상을 놓치는 경우 많음

Outlier Detection 접근 방식 2

2. Semi-supervised Outlier Detection

- 대부분의 현실 상황은 정상 데이터만 존재
- 정상 데이터로 모델 학습 -> 정상과 다르면 이상으로 판단
- 학습은 비정상 데이터를 보지 않고 진행됨

outlier Detection 접근 방식 3

3. Unsupervised Outlier Detection

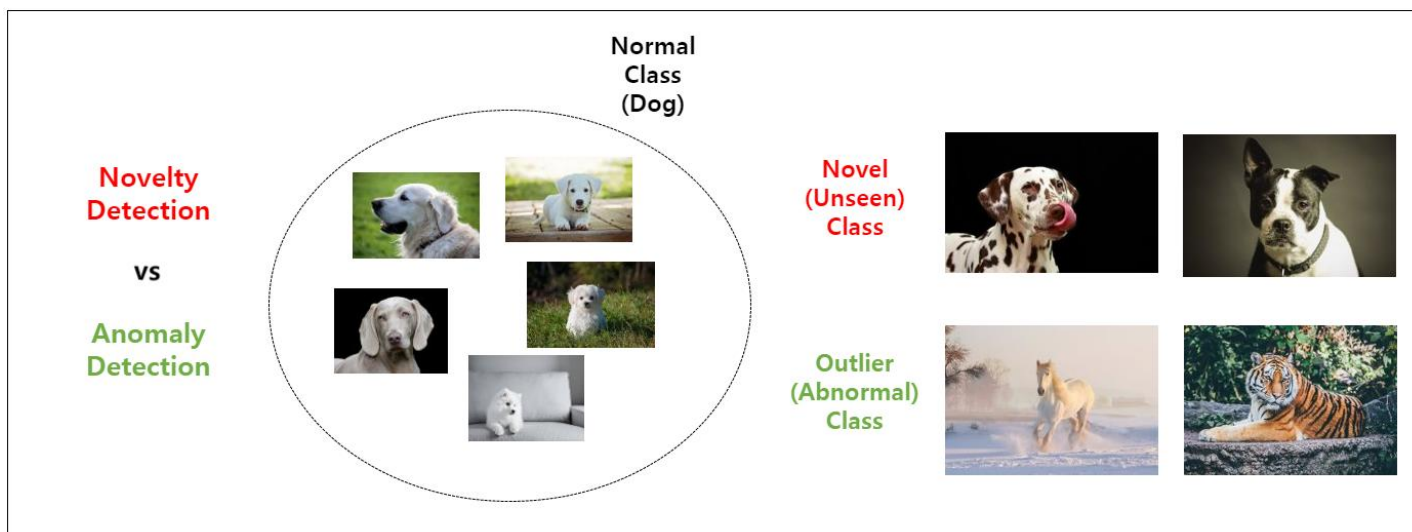
- 라벨링이 전혀 없는 데이터셋이 정상/이상을 스스로 구분
- 클러스터링, 거리 기반, 밀도 기반 기법 사용
- 이상 데이터는 **분포상 자연스럽게 않은 점**으로 간주

outlier Detection의 전제조건

- 대부분의 데이터는 정상(normal)이라 가정
- 비정상 데이터는 드물고 불균형한 구조
- 결정 기준:
 - 비정상 데이터가 충분한가? -> Supervised
 - 거의 없다면? -> Semi / Unsupervised

Outlier, Noise, Novelty

- Outlier: 통계적 기준에서 벗어난 **이상값** (이례적, 극단적)
- Noise: 의미 없는 **무작위 오류** (센서의 잡음, 전송 오류 등)
- Novelty: 훈련 데이터에 없는 **새로운 유형** (신규 고객 등)



Novelty Detection

- 훈련 데이터 외부의 새로운 유형을 이상으로 탐지
- 분포 밖 데이터(OOD)를 다루는 데 강점
 - 예: 기존 구매 이력과 전혀 다른 새로운 고객 세그먼트 등장,
시스템 업그레이드 후 새 로그 형식이 기존과 다른 경우

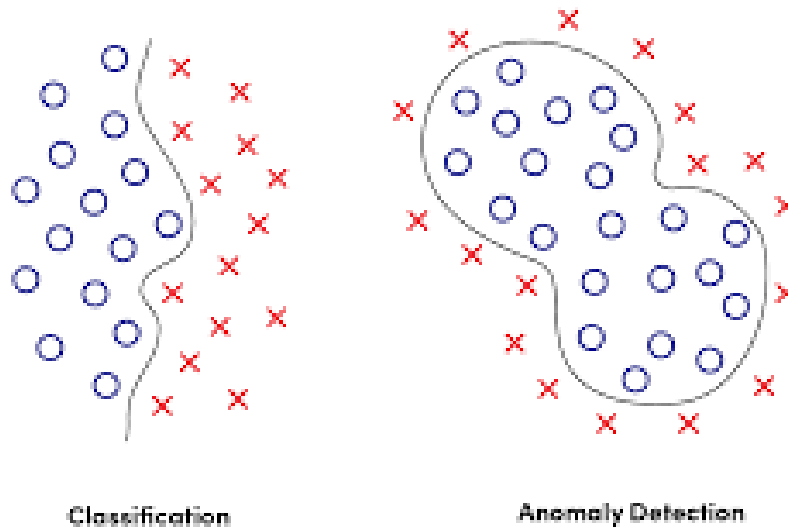
성능 평가 지표

- Accuracy는 다수 클래스에 편향될 수 있음
- 실제 이상 탐지에서는 precision, recall, f1-score가 중요
 - 예: 실제 비정상 - 100건 / 전체 10,000건
100% 정상 예측 -> Accuracy 99%
하지만 Recall 0%, 실제 탐지는 0건

Outlier Detection: Supervised Learning

지도학습 기반 이상치 탐지

- 정상과 비정상에 대한 **라벨이 사전에 주어진 상태**에서 모델이 패턴을 학습하고, 새로운 데이터가 이상인지 여부를 판별하는 방식이다.

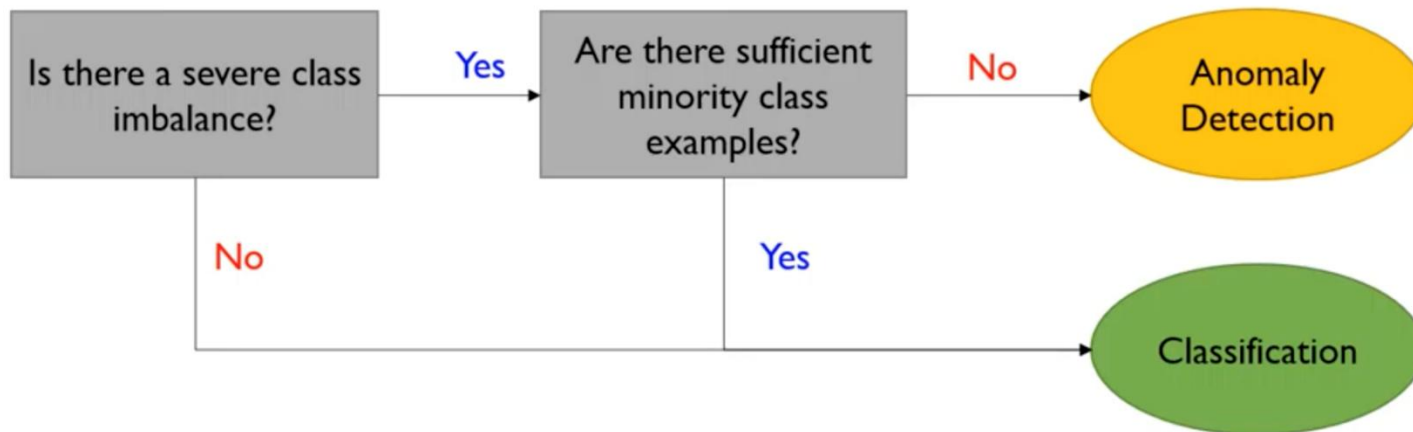


Outlier Detection vs. Classification

- 지도학습 기반 이상치 탐지는 분류 문제에 비하여 데이터 불균형이 있고, 소수 클래스 정보가 충분하지 않다.

Classification vs. Anomaly Detection

✓ Which one to use?



KNN 기반 이상 탐지

- KKN은 데이터 간의 거리를 측정하여 k개의 주변 이웃과의 유사도를 기반으로 분류하는 알고리즘
- 이상 탐지에서는 정상 데이터와 멀리 떨어진 포인트를 이상치로 간주

KNN 기반 이상 탐지 in Python

- 이상치를 포함하는 가상의 데이터 생성하여 라벨 부여

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsClassifier

# 이상치가 포함된 가상의 데이터 생성
np.random.seed(42)
normal_data = np.random.randn(300, 2) # 정규분포를 따르는 정상 데이터 300개
anomalies = 4 + 1.5 * np.random.randn(10, 2) # 정상 데이터에서 멀리 떨어진 이상치 10개 생성

# 정상 데이터와 이상치 데이터를 합침
data = np.vstack([normal_data, anomalies])

# 데이터에 라벨 부여 (정상: 1, 이상치: 0)
labels = np.concatenate([np.ones(len(normal_data)), np.zeros(len(anomalies))])
```

KNN 기반 이상 탐지 in Python

- KNN 학습 및 예측

```
# KNN 모델 초기화 및 학습
k = 5 # 이웃의 개수 설정
clf = KNeighborsClassifier(n_neighbors=k)
clf.fit(data, labels)

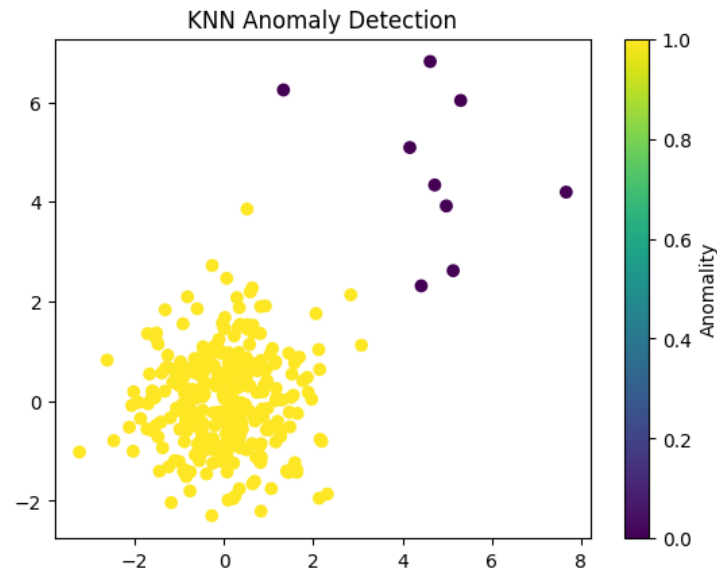
# 각 데이터 포인트가 이상치인지 예측 (0: 이상치, 1: 정상)
predictions = clf.predict(data)

# 이상치로 분류된 데이터 인덱스 출력
anomalies_indices = np.where(predictions == 0)[0]
print("탐지된 이상치 인덱스:", anomalies_indices)
```

KNN 기반 이상 탐지 in Python

■ KNN 결과 시각화

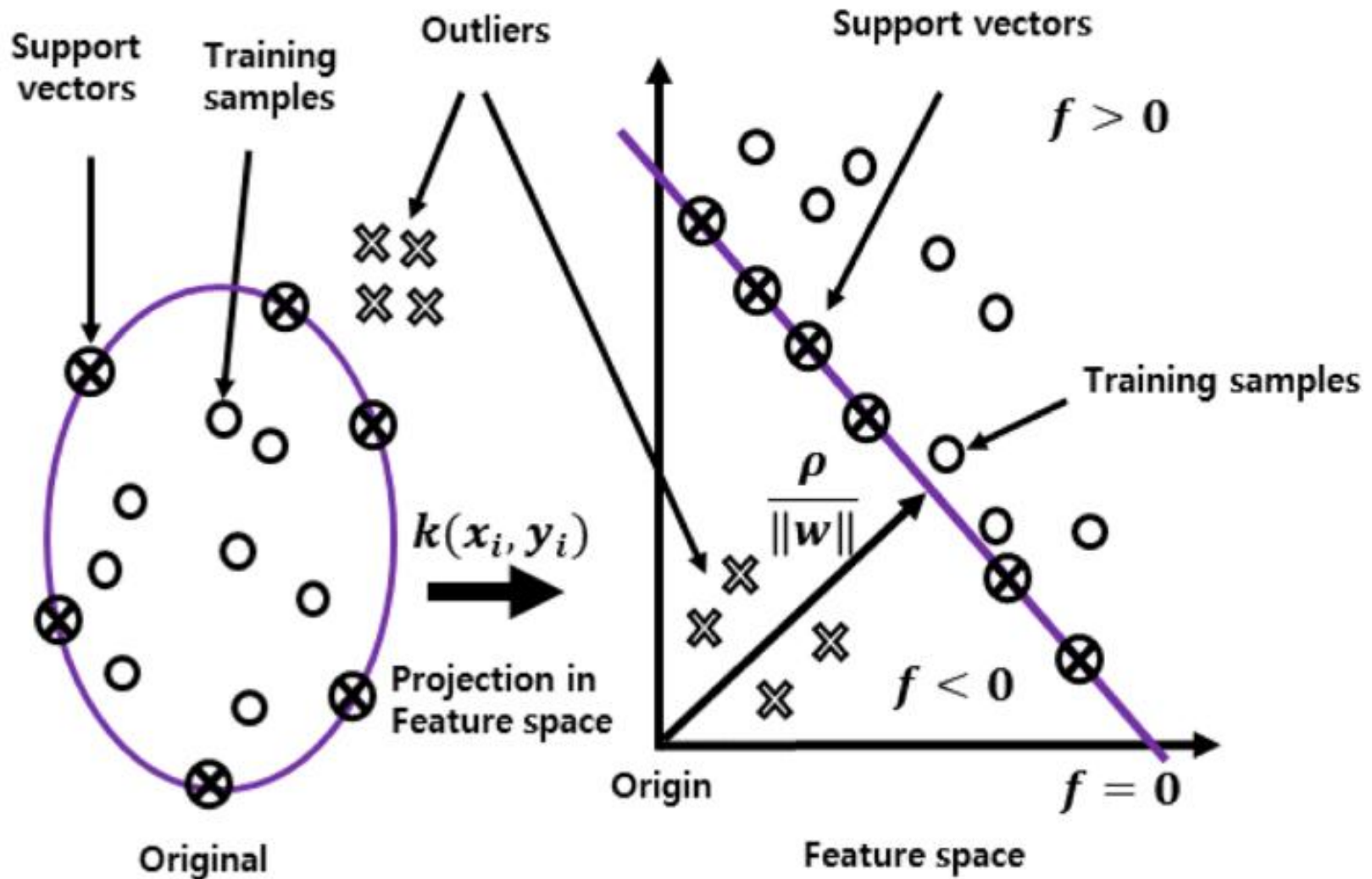
```
# 결과 시각화
plt.scatter(data[:, 0], data[:, 1], c=predictions, cmap='viridis')
plt.colorbar(label="Anomaly")
plt.title("KNN Anomaly Detection")
plt.show()
```



One-Class SVM 기반 이상 탐지

- One-Class SVM은 정상 데이터만으로 학습하여 **초평면 (Hyperplane)** 기반의 결정 경계를 학습한 뒤, 경계 밖의 포인트를 이상치로 분류
- 지도학습의 틀을 갖추고 있으나, 이상 데이터에 대한 정보 없이 정상 데이터의 분포를 기반으로 이상치를 탐지하므로 비지도학습으로도 간주할 수
- 소수의 이상 데이터만 존재하는 상황에서 효과적

One-Class SVM 기반 이상 탐지



One-Class SVM in Python

- 데이터 생성

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.svm import OneClassSVM

n_features = 2
n_outliers = 10

X, _ = make_blobs(n_samples=200, n_features=2, centers=1, random_state=42)
X[:n_outliers] = 5 + 2 * np.random.randn(n_outliers, n_features)
```

- 1-Class SVM 모델 학습

```
ocsvm = OneClassSVM(nu=0.05, kernel="rbf") # nu: 이상치 비율
ocsvm.fit(X)
labels_ocsvm = ocsvm.predict(X)

normal_data = X[labels_ocsvm == 1]
outliers = X[labels_ocsvm == -1]
```

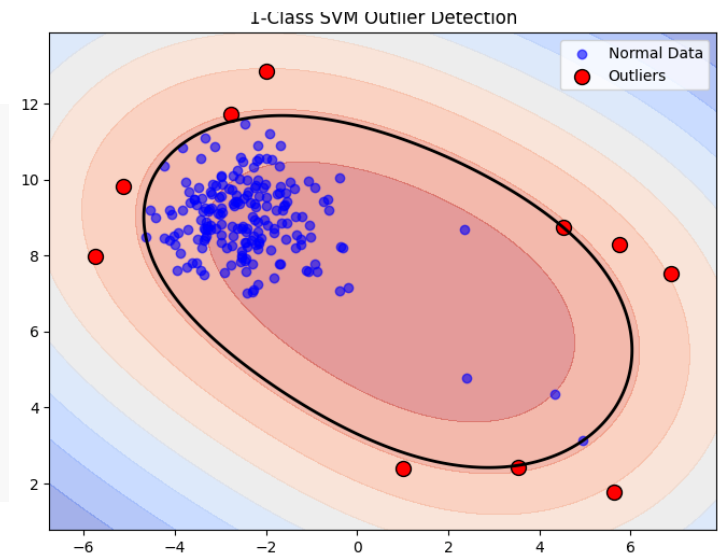
One-Class SVM in Python

- 결정 경계(decision boundary)를 시각화

```
xx, yy = np.meshgrid(np.linspace(X[:,0].min() - 1, X[:, 0].max() + 1, 500),  
                     np.linspace(X[:,1].min() - 1, X[:, 1].max() + 1, 500))  
Z = ocsvm.decision_function(np.c_[xx.ravel(), yy.ravel()])  
Z = Z.reshape(xx.shape)
```

- 결과 시각화

```
plt.figure(figsize=(8, 6))  
plt.contourf(xx, yy, Z, levels=np.linspace(Z.min(), Z.max(), 10),  
             cmap=plt.cm.coolwarm, alpha=0.5)  
plt.scatter(normal_data[:,0], normal_data[:,1],  
            label="Normal Data", color="blue", alpha=0.6)  
plt.scatter(outliers[:,0], outliers[:,1],  
            label="Outliers", color="red", edgecolors="k", s=100)  
plt.contour(xx, yy, Z, levels=[0], linewidths=2, colors="black")  
plt.legend()  
plt.title("1-Class SVM Outlier Detection")  
plt.show()
```



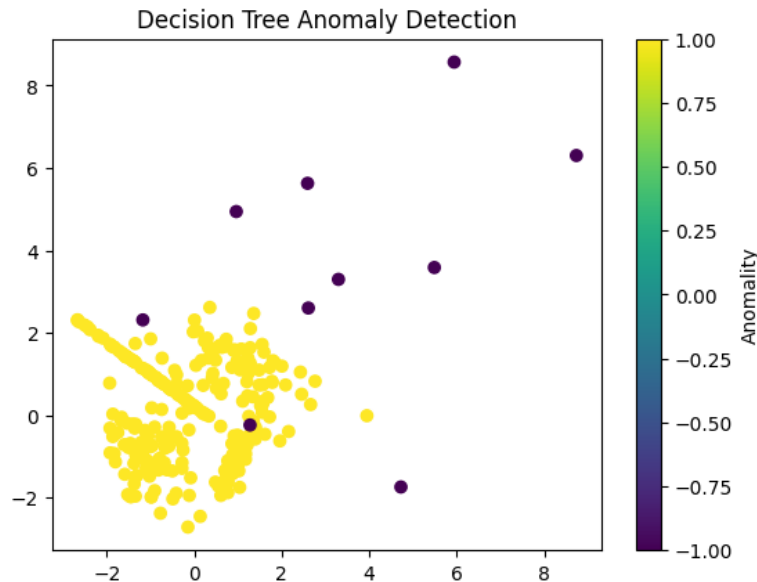
Decision Tree 기반 이상 탐지

- Decision Tree는 데이터를 feature의 임계값에 따라 분기하여 정상과 이상을 구분하는 규칙 기반 분류 모델
- 이상치는 깊은 노드에 존재하는 경우가 많음
- 트리 구조가 해석 가능하고 시각화 용이

Decision Tree in Python

- Decision Tree 학습 및 예측 결과 시각화

```
# Decision Tree 기반 이상치 탐지  
dt = DecisionTreeClassifier(random_state=42)  
dt.fit(data, labels)  
dt_predictions = dt.predict(data)
```



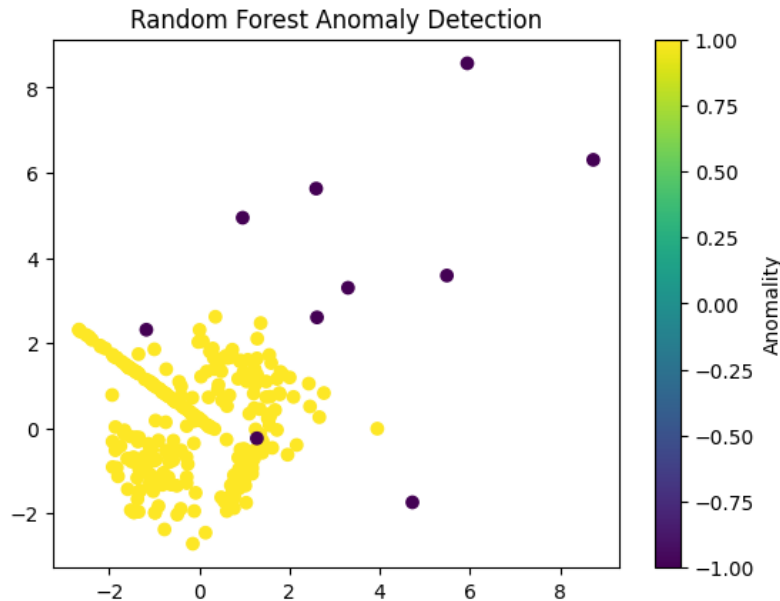
Random Forest 기반 이상 탐지

- 여러 개의 Decision Tree를 앙상블로 구성하여 voting을 통해 예측을 수행하는 모델
- 각 트리가 서로 다른 feature subset을 사용하여 학습
- 과적합 방지 효과가 있음

Random Forest in Python

- Random Forest 학습 및 예측 결과 시각화

```
# Random Forest 기반 이상치 탐지  
rf = RandomForestClassifier(n_estimators=100, random_state=42)  
rf.fit(data, labels)  
rf_predictions = rf.predict(data)
```



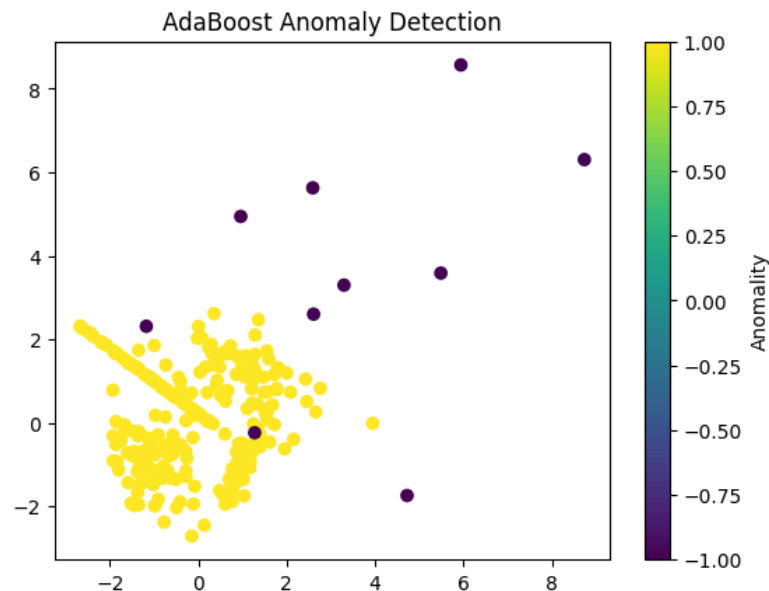
AdaBoost 기반 이상 탐지

- AdaBoost는 여러 개의 weak learners를 조합하여 오분류된 샘플에 가중치를 더해 다음 학습기에 전달하는 방식
- 이상치처럼 잘못 예측된 데이터에 더 집중
- 깊이 1의 트리를 기본 학습기로 자주 사용

AdaBoost in Python

- AdaBoost 학습 및 예측 결과 시각화

```
# AdaBoost 기반 이상치 탐지  
ada = AdaBoostClassifier(n_estimators=100, random_state=42)  
ada.fit(data, labels)  
ada_predictions = ada.predict(data)
```



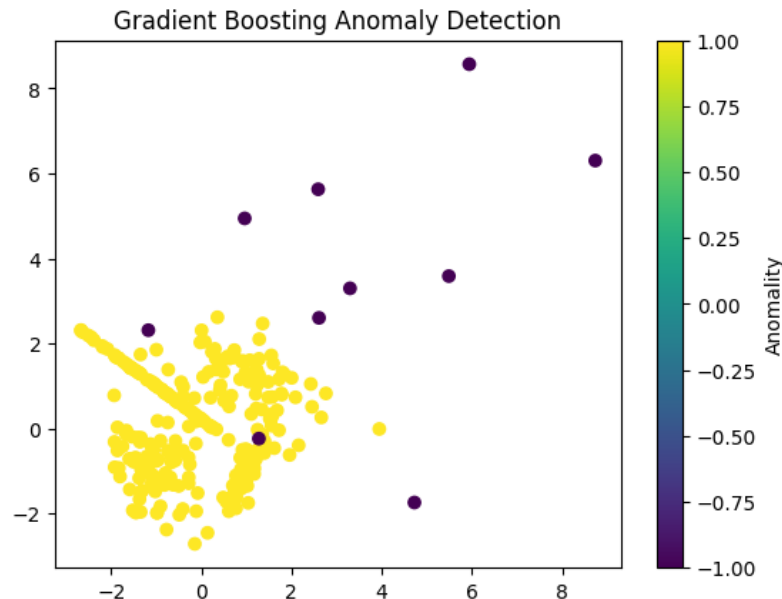
Gradient Boosting 기반 이상 탐지

- Gradient Boosting은 오차를 반복적으로 보정해나가며 모델을 개선하는 순차적 앙상블 학습 방식
- 오차를 줄이는 방향으로 학습
- 다양한 손실 함수에 적용 가능
- 이상치 탐지 시 정밀도 높은 예측이 가능

Gradient Boosting in Python

- Gradient Boosting 학습 및 예측 결과 시각화

```
# Gradient Boosting 기반 이상치 탐지  
gb = GradientBoostingClassifier(n_estimators=100, random_state=42)  
gb.fit(data, labels)  
gb_predictions = gb.predict(data)
```



XGBoost 기반 이상 탐지

- XGBoost는 Gradient Boosting의 확장 버전으로 정규화, 병렬 처리, 결측값 처리 등 성능을 극대화한 알고리즘
- 실무에서 가장 널리 쓰이는 모델
- Feature 중요도 분석 가능

XGBoost 기반 이상 탐지 in Python

- 이상치를 포함하는 가상의 데이터 생성하여 라벨 부여 및 학습/테스트 데이터 분할

```
# 이상치가 포함된 가상의 데이터 생성
np.random.seed(42)
X, y = make_classification(n_samples=500, n_features=2, n_classes=2, n_clusters_per_class=1, n_redundant=0)

# 이상치 데이터 생성 (정상 데이터와 멀리 떨어진 위치에 생성)
anomalies = 2 + 0.5 * np.random.randn(10, 2)

# 원본 데이터와 이상치 데이터 결합
X = np.vstack([X, anomalies])
y = np.hstack([y, np.ones(len(anomalies))]) # 이상치는 라벨 1, 정상 데이터는 라벨 0

# 학습 데이터와 테스트 데이터로 분할
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

XGBoost in Python

- XGBoost 학습 및 예측 결과 출력

```
# XGBoost를 이용한 이상치 탐지
xgb = XGBClassifier()
xgb.fit(X_train, y_train)
y_xgb_pred = xgb.predict(X_test)
```

```
print(f"모델: XGBoost")
print("혼동 행렬 (Confusion Matrix):")
print(confusion_matrix(y_test, y_pred))
print("분류 성능 지표 (Classification Report):")
print(classification_report(y_test, y_pred))
print("="*50)
```

Naïve Bayes 기반 이상 탐지

- Naïve Bayes는 각 피처가 독립이라는 가정 하에 확률적 접근 방식으로 분류를 수행하는 알고리즘
- 입력값이 주어졌을 때 정상 확률 vs 이상 확률을 비교
- 이상치 탐지에서는 확률이 매우 낮은 샘플을 이상으로 간주

```
# Naive Bayes를 이용한 이상치 탐지
nb = GaussianNB()
nb.fit(X_train, y_train)
y_nb_pred = nb.predict(X_test)
```

Logistic Regression 기반 이상 탐지

- 로지스틱 회귀는 이진 분류 문제에 가장 널리 사용되는 선형 모델로, 이상 탐지에서는 결정 경계(sigmoid function)를 기반으로 이상 여부 분류
- Feature 값에 따라 이상일 확률 계산

```
# Logistic Regression을 이용한 이상치 탐지
lr = LogisticRegression()
lr.fit(X_train, y_train)
y_lr_pred = lr.predict(X_test)
```

EVT 기반 이상 탐지

- Extreme Value Theory는 데이터 분포의 극단값(꼬리 부분)만을 모델링하여 드물게 발생하는 이상값 탐지
- 극단적인 값이 나타날 확률 기반 판단
- 확률 밀도 함수 기반으로 임계값 결정

EVT in Python

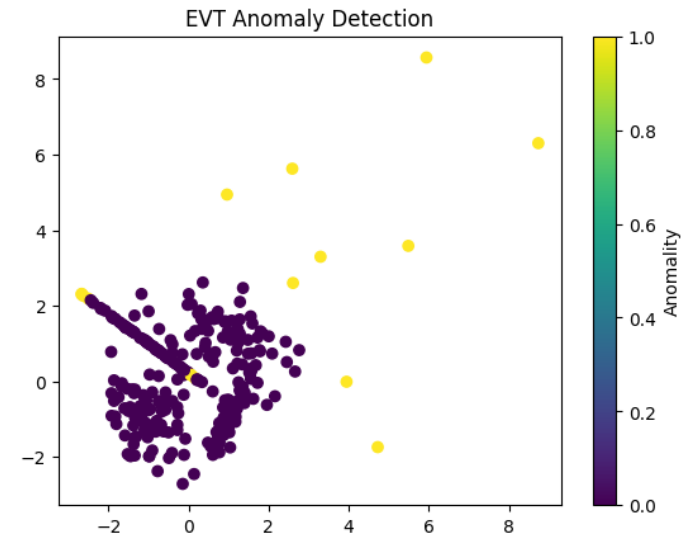
■ EVT 학습 및 예측 결과 시각화

```
from sklearn.preprocessing import StandardScaler

# 1. 스케일링 (정규분포에 맞게 스케일 조정)
scaler = StandardScaler()
scaled_data = scaler.fit_transform(data)

# 2. 각 데이터 포인트의 L2 노름(거리) 계산
scores = np.linalg.norm(scaled_data, axis=1)

# 3. EVT 모델링: 정규분포 적합 후 threshold 설정
mu, sigma = norm.fit(scores)
pdf_vals = norm.pdf(scores, mu, sigma)
threshold = np.percentile(pdf_vals, 5) # 하위 5%를 이상치로 간주
evt_predictions = (pdf_vals < threshold).astype(int)
```



모델 선택 가이드

- 사용할 목적 및 데이터에 따라 모델 선택 필요

모델	특징	특징
KNN	거리 기반, 간단	소규모, 저차원 데이터
SVM	경계 기반, 고정밀	비선형 경계 필요시
RF/GBM	트리 기반 앙상블	해석보단 성능 중시
XGBost	고성능 엔진	대규모 이상 탐지

지도학습 기반 이상 탐지 마무리

- 지도학습 기반 이상 탐지는 **풍부한 라벨링 데이터**가 필요
- 모델 선택은 데이터의 구조, 복잡성, 이상 탐지의 목적, 해석력 필요 여부에 따라 달라짐
- **불균형 데이터 대응 전략**이 핵심적
- 단일 모델보다 앙상블 및 후처리 기반 혼합 전략이 효과적일 수 있음

Lab 6-1: Supervised Outlier Detection

Lab

- NH Data Science Course

- All code examples are written in Python 3
- Please do not hesitate to ask questions!
- TAs
 - 김규태
 - 최예림
- 모든 Lab 자료는 eX-campus에 업로드 되어 있습니다.

Outlier Detection: Unsupervised Learning

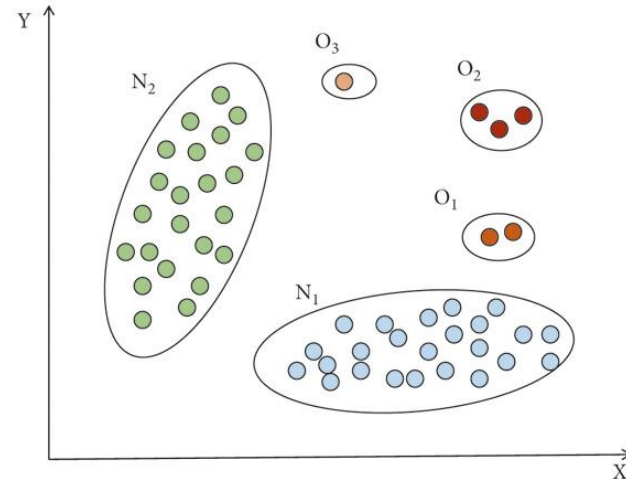
비지도 이상 탐지

- 사전에 어떤 데이터가 정상인지 이상치인지 알지 못한 상태에서, 데이터의 패턴을 분석하여 이상치를 탐지하는 방법
- 지도 이상 탐지(Supervised Outlier Detection)나 준지도 이상 탐지(Semi-supervised Outlier Detection)는 반드시 정상 샘플에 대한 라벨이 필요함
- 데이터의 정상 여부조차 파악되지 않는 상황이라면?
→ 비지도 학습 기법 활용



비지도 이상 탐지

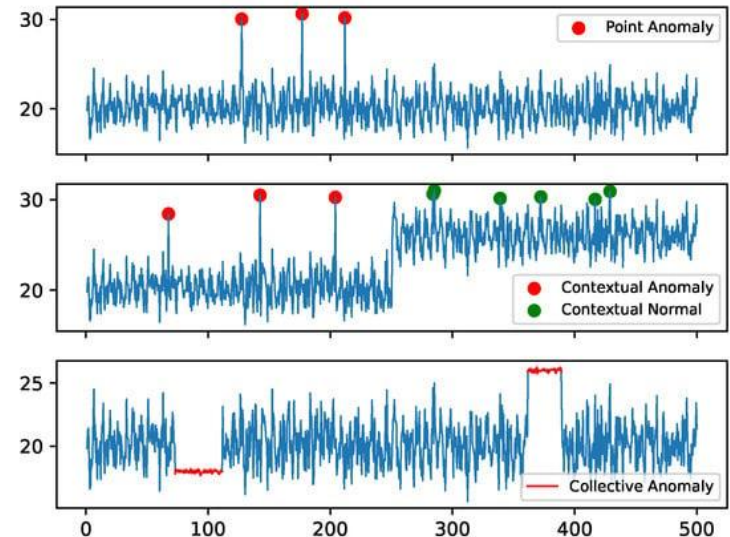
- 대부분의 데이터가 정상 샘플이라는 가정하에 학습하기 때문에, 정상 여부에 대한 라벨이 필요하지 않음
 - 정상 데이터와 이상 데이터의 분포를 몰라도 동작 가능
 - 현실 적용 가능성 높음
- 특정한 이상치 유형에 국한되지 않고 새로운 유형의 이상치를 탐지할 가능성이 높음



비지도 이상 탐지

[비지도 이상 탐지 유의점]

- 모델 학습 후, 정상/비정상 데이터의 구분에 대한 임계치 설정이 필요할 수 있음
 - 정상과 이상 데이터를 명확히 구분하는 기준이 없기 때문에, **False Positive**(정상 값을 이상치로 판단) 가능성 존재
- 데이터 분포에 따라 이상 탐지 성능이 크게 달라질 수 있음

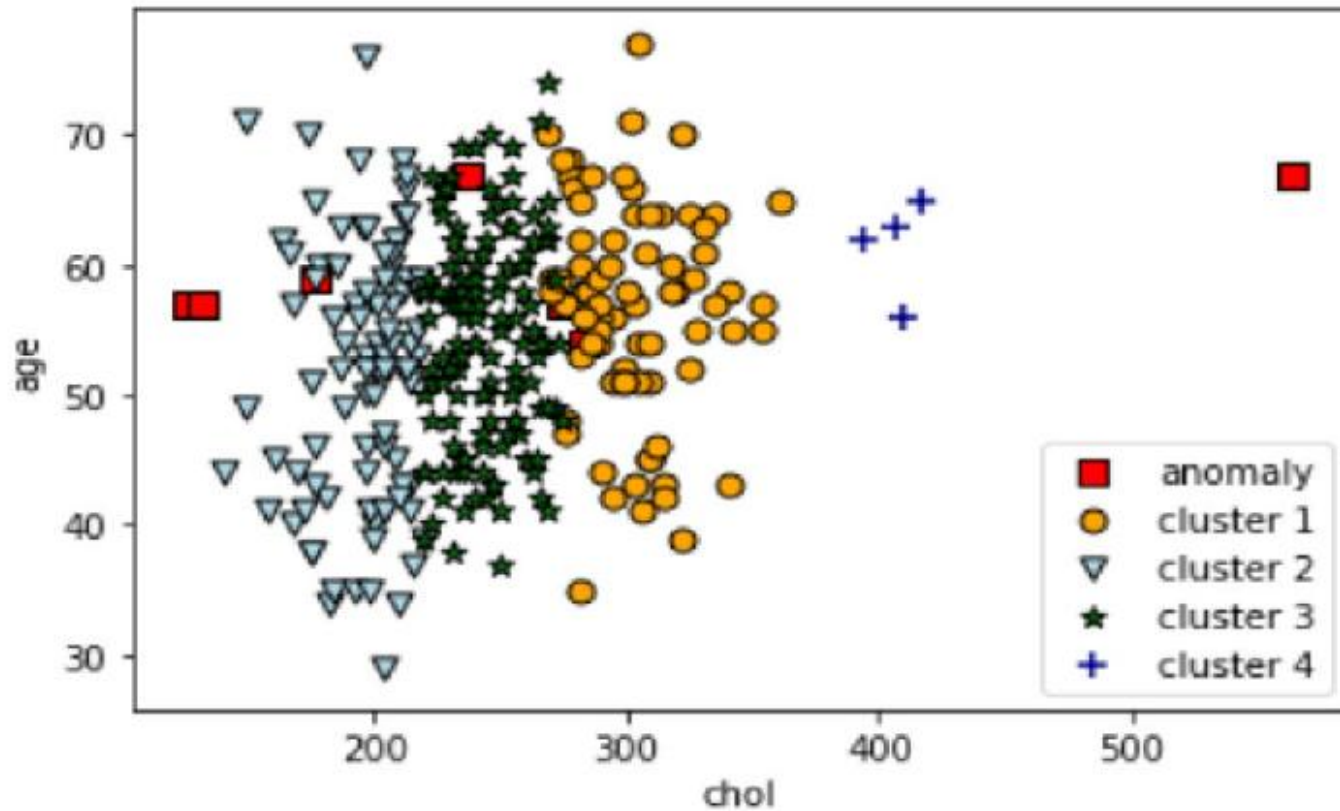


Distance/Density-Based Approaches

K-Means 이상 탐지

- 특정 데이터 포인트가 속한 클러스터 중심과의 거리를 계산하여 멀리 떨어진 데이터 포인트를 이상치로 탐지
- 이상 탐지 방식
 - 데이터를 k개의 클러스터로 분류
 - 각 데이터 포인트와 클러스터 중심 사이의 거리 계산
 - 거리가 일정 임계값(threshold) 이상이면 이상치로 판단

K-Means



K-Means 이상 탐지

장점

- 밀도 기반 방법(DBSCAN 등)보다 계산량이 적음

단점

- 적절한 클러스터 개수 k 를 설정하는 것이 필요
- 이상치가 많은 경우 클러스터 중심이 이상치에 의해 왜곡될 수 있음

K-Means 이상 탐지 in Python

- 사이킷런(scikit-learn)의 KMeans 클래스 사용

<KMeans 클래스 생성자 파라미터>

Constructor parameter	Default	Example values	Description
<code>n_clusters</code>	8	3, 36	The number of clusters to determine
<code>max_iter</code>	300	200, 37	Maximum number of iterations for a single run
<code>n_init</code>	10	8, 10	Number of times to rerun the algorithm with different seeds
<code>tol</code>	1e-4	1e-3, 1e-2	Value regulating stopping conditions

K-Means 이상 탐지 in Python

- 랜덤 2D 샘플 데이터 생성

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

X, _ = make_blobs(n_samples=300, centers=3, cluster_std=1.0, random_state=42)
```

- K-Means 모델 학습 (군집 개수 K=3)

```
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
kmeans.fit(X)
labels = kmeans.labels_ # 각 데이터의 군집 라벨
centroids = kmeans.cluster_centers_ # 군집 중심 좌표
distances = np.linalg.norm(X - centroids[labels], axis=1) # 데이터와 군집 중심 간 거리
```

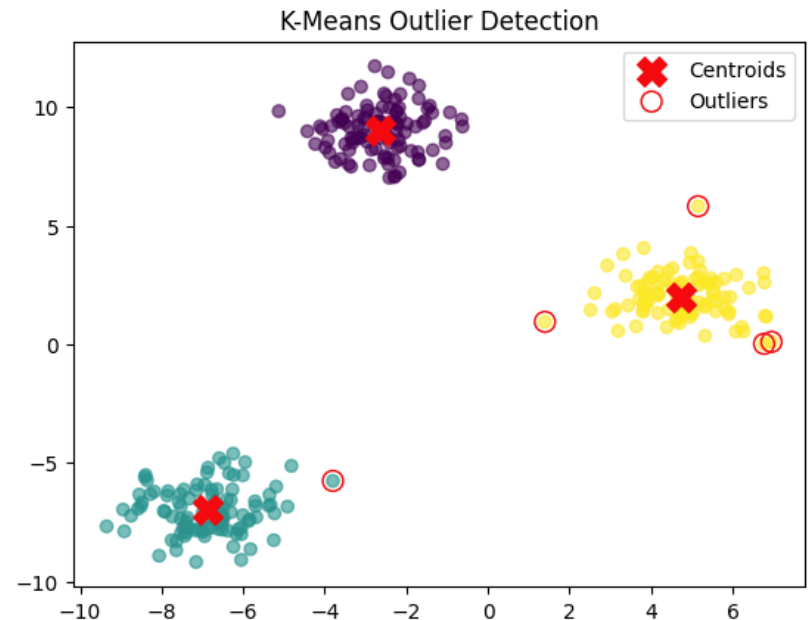
K-Means 이상 탐지 in Python

- 이상치 탐지 (거리가 평균 + 2.5 표준편차 이상인 데이터)

```
threshold = np.mean(distances) + 2.5 * np.std(distances)
outliers = X[distances > threshold]
```

- 결과 시각화

```
plt.scatter(X[:,0], X[:,1], c=labels, cmap='viridis', alpha=0.6)
plt.scatter(centroids[:,0], centroids[:,1], marker='X', s=200,
           c='red', label="Centroids")
plt.scatter(outliers[:,0], outliers[:,1], marker='o', s=100,
           edgecolors='r', facecolors='none', label="Outliers")
plt.legend()
plt.title("K-Means Outlier Detection")
plt.show()
```



DBSCAN 이상 탐지

- DBSCAN Density-Based Spatial Clustering of Applications with Noise
: 밀도를 기반으로 클러스터를 형성하고, 밀도가 낮은 지역에 있는 데이터 포인트를 이상치로 탐지
 - 이상 탐지 방식
 - 특정 반경(ϵ) 내에 포함된 데이터 포인트 수 측정
 - 이 수가 최소 임계값(MinPts)보다 작으면 이상치로 간주
- ✓ ϵ 와 MinPts 설정이 어렵지만, 비선형 구조 데이터에 적합함

DBSCAN 이상 탐지

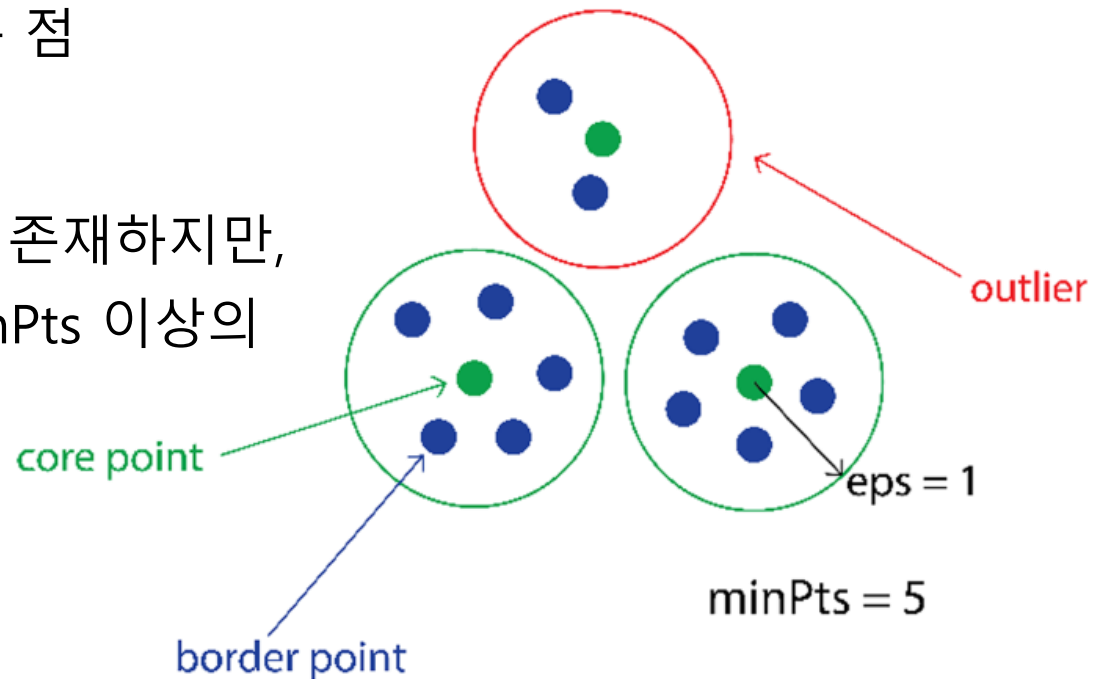
- Core Point

: 반경 ε 내에 최소 MinPts 개수 이상의 데이터 포인트가 존재하는 점

- Border Point

: Core Point의 ε 반경 내에 존재하지만, 자기 자신을 중심으로 MinPts 이상의 이웃이 없는 포인트

- Outlier(=Noise Point)



DBSCAN 이상 탐지 in Python

- 랜덤 반달형 데이터 생성

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import DBSCAN
from sklearn.datasets import make_moons

X, _ = make_moons(n_samples=300, noise=0.2, random_state=123)
```

- DBSCAN 모델 학습 (반경 $\epsilon=0.2$, 최소 샘플 수 MinPts=3)

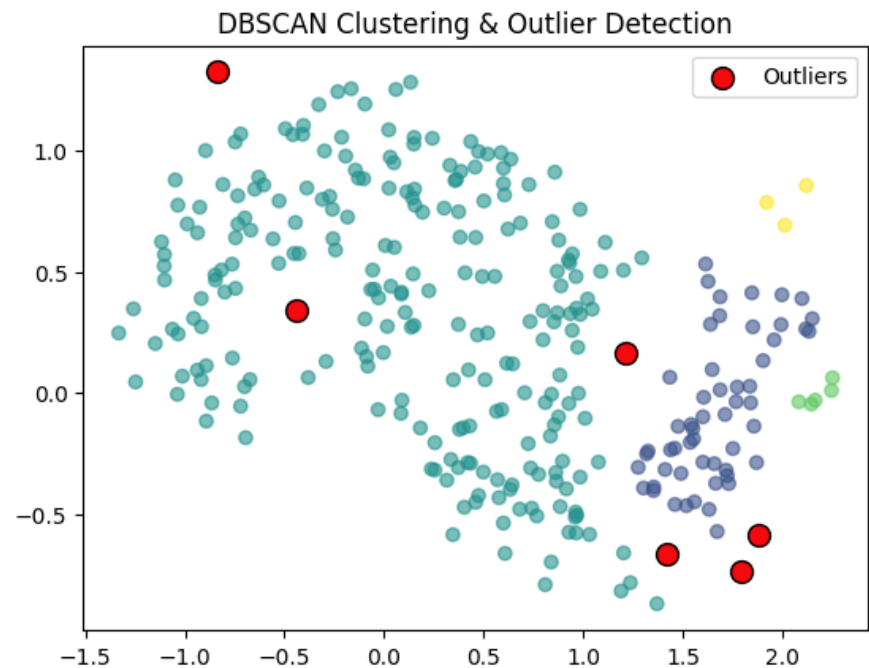
```
dbscan = DBSCAN(eps=0.2, min_samples=3)
labels = dbscan.fit_predict(X)

outliers = X[labels == -1] # 이상치(Noise) 확인 (-1로 라벨링됨)
```

DBSCAN 이상 탐지 in Python

■ 결과 시각화

```
plt.scatter(X[:,0], X[:,1], c=labels,  
            cmap='viridis', alpha=0.6)  
plt.scatter(outliers[:,0], outliers[:,1], color='red',  
            edgecolors='k', s=100, label="Outliers")  
plt.title("DBSCAN Clustering & Outlier Detection")  
plt.legend()  
plt.show()
```



LOF 이상 탐지

- LOF Local Outlier Factors

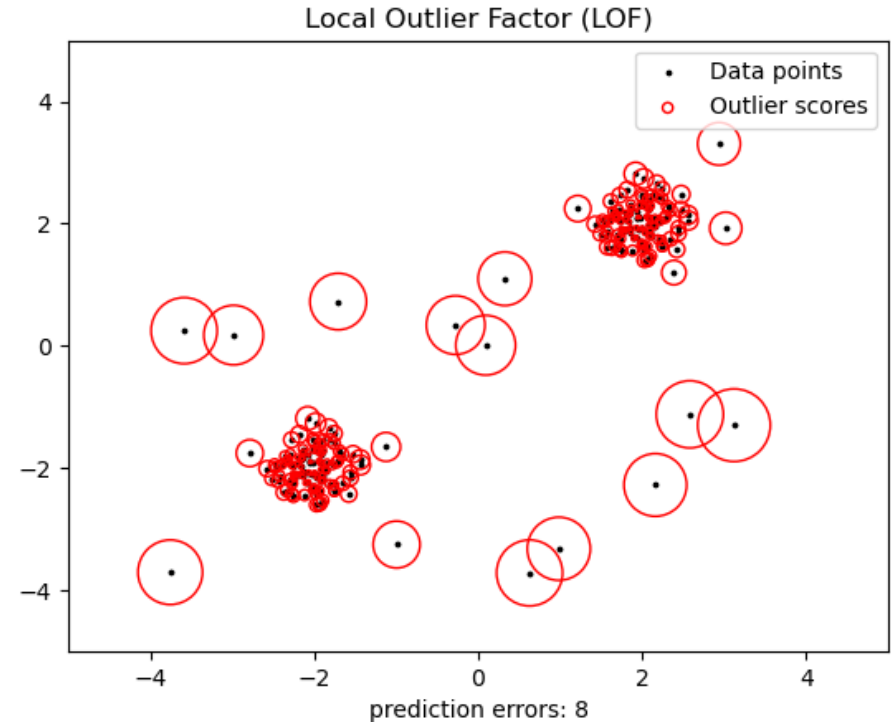
: 데이터 포인트의 지역적 밀도를 계산하여, 밀도가 낮은 데이터를 이상치로 탐지

- 이상 탐지 방식

- 각 데이터 포인트에 대해 k개의 최근접 이웃을 찾음
- 데이터 포인트의 밀도를 측정하고, 이웃의 평균 밀도와 비교
- 밀도가 주변보다 현저히 낮으면 이상치로 간주

LOF 이상 탐지

- KNN과 비슷하지만,
단순 거리 기반이 아닌
'밀도' 기반으로 이상치 판별함
- 밀도가 낮은 데이터일수록
LOF 값이 큼
→ 클수록 이상치일 가능성 ↑



LOF 이상 탐지 in Python

- 랜덤 반달형 데이터 생성

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.neighbors import LocalOutlierFactor

n_features = 2
n_outliers = 10

X, _ = make_blobs(n_samples=200, n_features=2, centers=1, random_state=42)
X[:n_outliers] = 5 + 2 * np.random.randn(n_outliers, n_features) # 이상치 추가
```

LOF 이상 탐지 in Python

- LOF 모델 학습 (이웃 개수 $k=13$)

```
lof = LocalOutlierFactor(n_neighbors=13, contamination=0.01)
y_pred = lof.fit_predict(X)
lof_scores = -lof.negative_outlier_factor_ # LOF 점수 (높을수록 이상치 가능성↑)
```

- "contamination" Parameter

: 이상치(Outlier)의 비율을 지정하는 하이퍼파라미터

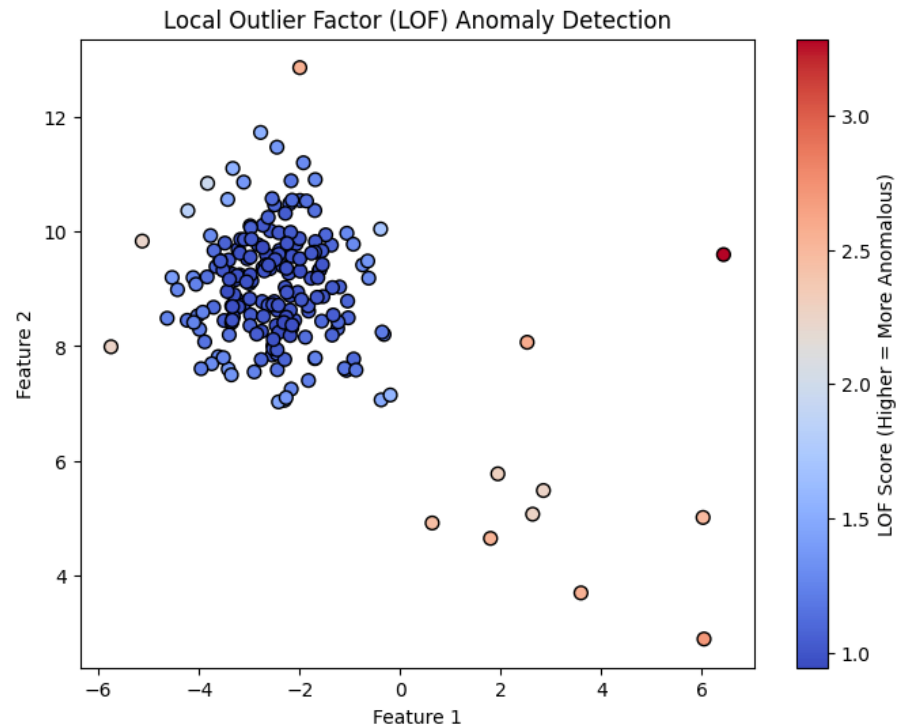
- 너무 작게 설정하면 실제 이상치를 놓칠 수 있음
- 너무 크게 설정하면 정상 데이터도 이상치로 잘못 판단할 수 있음

- 이상치 비율을 모른다면 → `contamination='auto'` 사용하여 자동 조정

LOF 이상 탐지 in Python

■ 결과 시각화

```
plt.figure(figsize=(8, 6))
scatter = plt.scatter(X[:,0], X[:,1], c=lof_scores,
                     cmap="coolwarm", edgecolors="k", s=50)
plt.colorbar(scatter, label="LOF Score (Higher = More Anomalous)")
plt.title("Local Outlier Factor (LOF) Anomaly Detection")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```



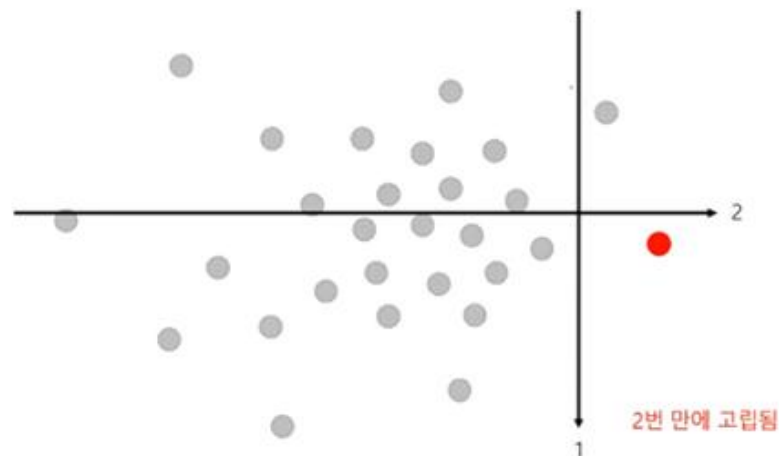
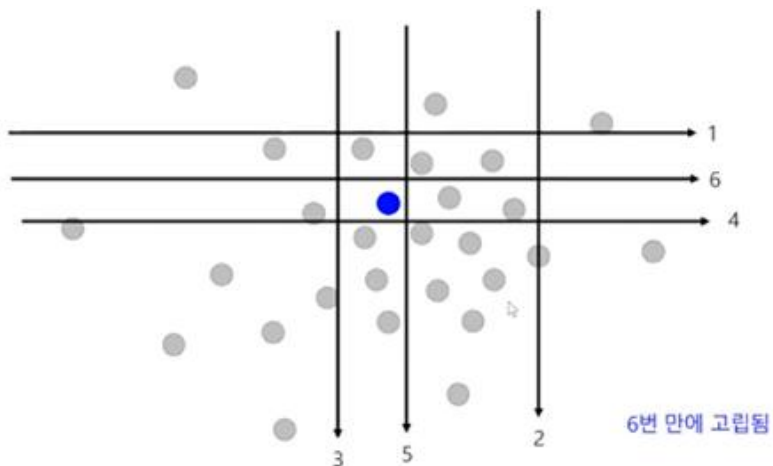
Model-Based Approaches

Isolation Forest 이상 탐지

■ Isolation Forest

: 여러 개의 의사결정나무(Decision Tree)를 종합한 앙상블 기반의 이상탐지 기법으로, 트리를 반복적으로 생성하며 샘플의 고립 정도 여부에 따라 이상치 탐지

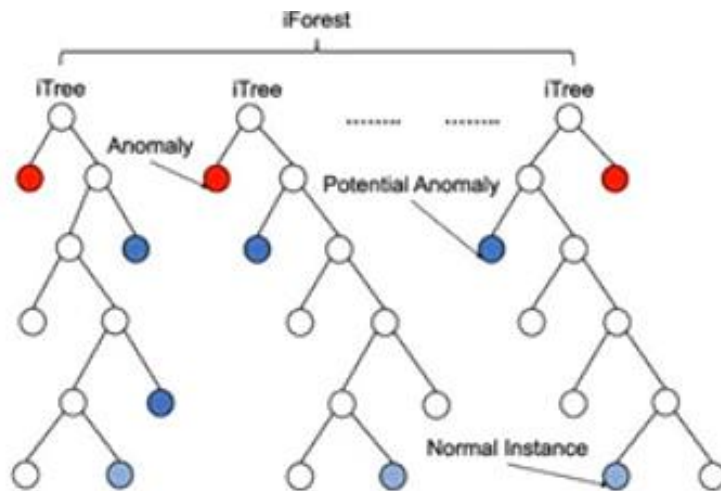
- 정상 샘플은 고립되기 어려울 것이라 가정 (이진 분할 多 수행)
- 이상치는 적은 분할 횟수로 고립



Isolation Forest 이상 탐지

■ 이상 탐지 방식

- 샘플을 랜덤하게 선택하고, 임의의 변수(splitting variable)와 분할점(splitting point)을 이용해 이진 분할을 반복하여 Isolation Tree 구축
- 각 데이터 포인트가 고립될 때까지의 분리 횟수를 기록
- 여러 개의 트리에서 평균 분리 횟수(path length)를 계산하여 outlier Score를 도출, 점수가 높을수록 이상치로 판별



Isolation Forest 이상 탐지 in Python

■ 데이터 생성

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.ensemble import IsolationForest

np.random.seed(42)
normal_data = np.random.randn(300, 2)
anomalies = 4 + 1.5 * np.random.randn(10, 2)
data = np.vstack([normal_data, anomalies])
```

■ Isolation Forest 모델 학습

```
clf = IsolationForest(contamination=0.05, random_state=42)
clf.fit(data)
predictions = clf.predict(data)
scores = clf.decision_function(data) # 이상치 점수 (클수록 정상, 작을수록 이상치)
```

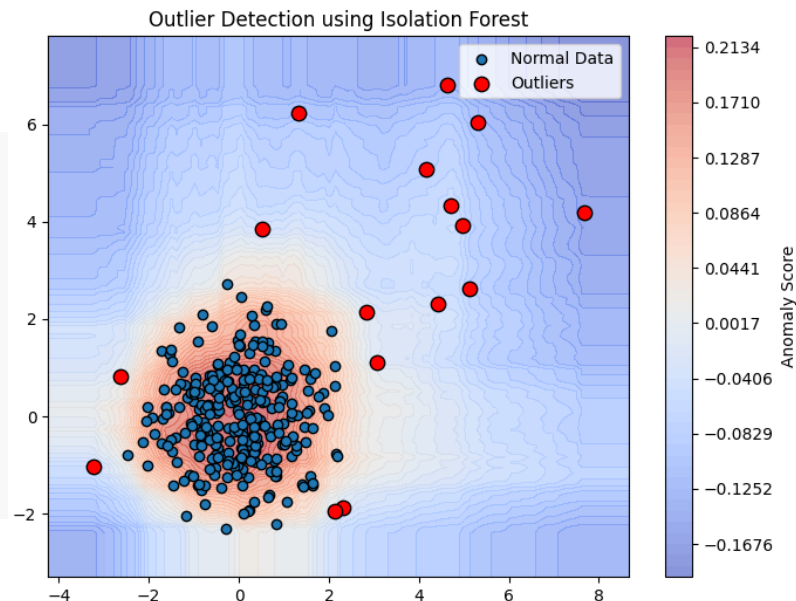
Isolation Forest 이상 탐지 in Python

- 결정 경계(decision boundary)를 시각화하기 위해 Grid 생성

```
xx, yy = np.meshgrid(np.linspace(data[:,0].min()-1, data[:,0].max()+1, 100),  
                     np.linspace(data[:,1].min()-1, data[:,1].max()+1, 100))  
Z = clf.decision_function(np.c_[xx.ravel(), yy.ravel()])  
Z = Z.reshape(xx.shape)
```

- 결과 시각화

```
plt.figure(figsize=(8, 6))  
plt.contourf(xx, yy, Z, levels=np.linspace(Z.min(), Z.max(), 50),  
             cmap='coolwarm', alpha=0.6) # Decision Boundary  
plt.colorbar(label="Anomaly Score")  
plt.scatter(data[predictions == 1, 0], data[predictions == 1, 1],  
            label="Normal Data", edgecolors="k")  
plt.scatter(data[predictions == -1, 0], data[predictions == -1, 1],  
            color="red", edgecolors="k", s=80, label="Outliers")  
plt.legend()  
plt.title("Outlier Detection using Isolation Forest")  
plt.show()
```



PCA 이상 탐지

- PCA Principal Component Analysis

: 데이터를 주성분(Principal Components)으로 변환하여 데이터를 저차원으로 압축했을 때, 주성분으로 설명되지 않는 작은 변동성을 가진 데이터를 이상치로 탐지

- 이상 탐지 방식

- 데이터를 주성분으로 변환하여 주요 정보를 추출
- 각 데이터 포인트를 주성분으로 재구성하고, 원래 데이터와의 차이(재구성 오차)를 계산
- 재구성 오차가 특정 임계값(threshold)을 초과하는 데이터 포인트를 이상치로 간주

PCA 이상 탐지 in Python

■ 데이터 생성

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA

np.random.seed(42)
X_normal = np.random.randn(300, 2)
X_outliers = np.random.uniform(-5, 5, size=(10, 2))
X = np.vstack([X_normal, X_outliers])
```

■ PCA 모델 학습

```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
```

PCA 이상 탐지 in Python

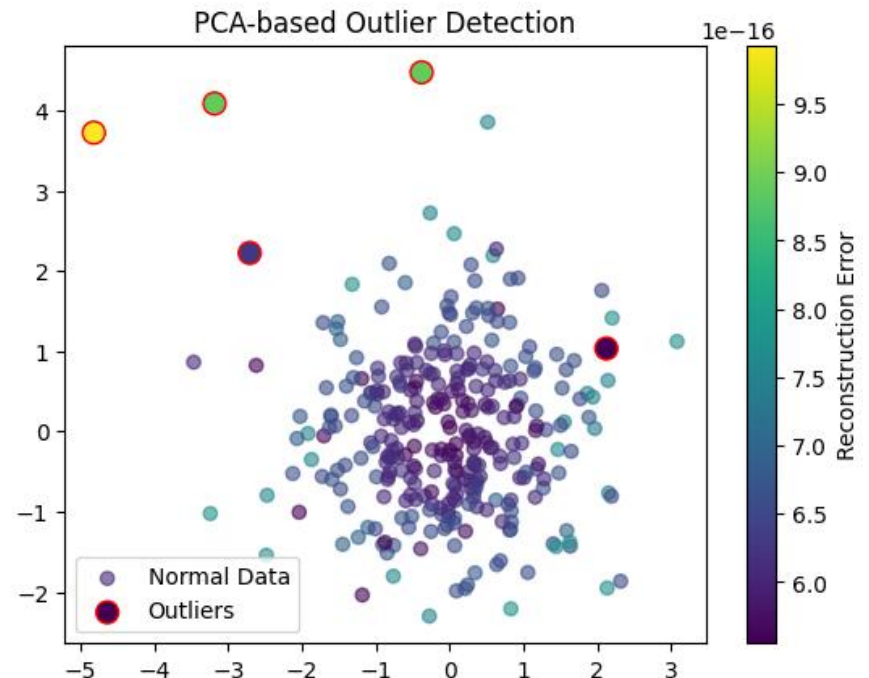
■ 재구성 오차 계산 및 임계값(threshold) 설정

```
X_reconstructed = pca.inverse_transform(X_pca)
recon_error = np.linalg.norm(X_scaled - X_reconstructed, axis=1)

threshold = np.percentile(recon_error, 97) # 상위 3%를 이상치로 간주
outliers = X[recon_error > threshold]
```

■ 결과 시각화

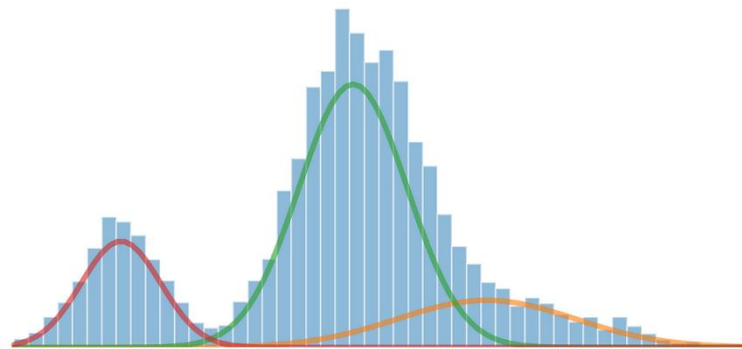
```
plt.scatter(X[:,0], X[:,1], c=recon_error,
            cmap='viridis', label="Normal Data", alpha=0.6)
plt.scatter(outliers[:,0], outliers[:,1], edgecolors='red',
            c=recon_error[recon_error > threshold],
            cmap='viridis', s=100, label="Outliers")
plt.colorbar(label="Reconstruction Error")
plt.legend()
plt.title("PCA-based Outlier Detection")
plt.show()
```



Gaussian Mixture Model 이상 탐지

■ Gaussian Mixture Model(GMM)

: 데이터가 여러 개의 가우시안 분포(정규 분포)로 구성된다
고 가정하고, 각 샘플이 속할 확률이 낮은 군집에 속하는 점
을 이상치로 탐지



■ 이상 탐지 방식

- GMM을 이용하여 데이터를 여러 개의 가우시안 분포로 모델링
- 각 데이터 포인트가 모델에 의해 생성될 확률(우도) 계산
- 확률이 매우 낮은 데이터 포인트는 이상치로 간주
(즉, 모델에서 설명되지 않는 데이터는 이상치로 판단)

GMM 이상 탐지 in Python

- 데이터 생성

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.mixture import GaussianMixture

normal_data = np.random.randn(300, 2)
anomalies = 4 + 1.5 * np.random.randn(10, 2)
data = np.vstack([normal_data, anomalies])
```

- GMM 모델 학습 (5개의 가우시안 분포)

```
n_components = 5
gmm = GaussianMixture(n_components=n_components,
                       covariance_type='full', random_state=42)
gmm.fit(data)
```

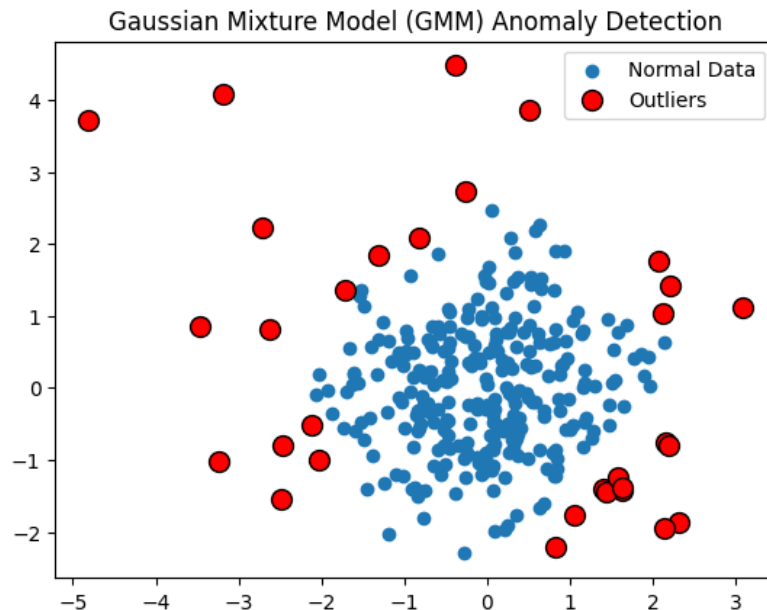
GMM 이상 탐지 in Python

- 각 샘플 확률 계산 및 확률 낮은 값 이상치 탐지

```
probs = gmm.score_samples(X)
threshold = np.percentile(probs, 10) # 확률 상위 10%를 정상으로, 나머지는 이상치로
outliers = X[probs < threshold]
```

- 결과 시각화

```
plt.scatter(X[:, 0], X[:, 1], label="Normal Data")
plt.scatter(outliers[:, 0], outliers[:, 1], color='red',
            edgecolors='k', s=100, label="Outliers")
plt.legend()
plt.title("Gaussian Mixture Model (GMM) Anomaly Detection")
plt.show()
```



Autoencoder 이상 탐지

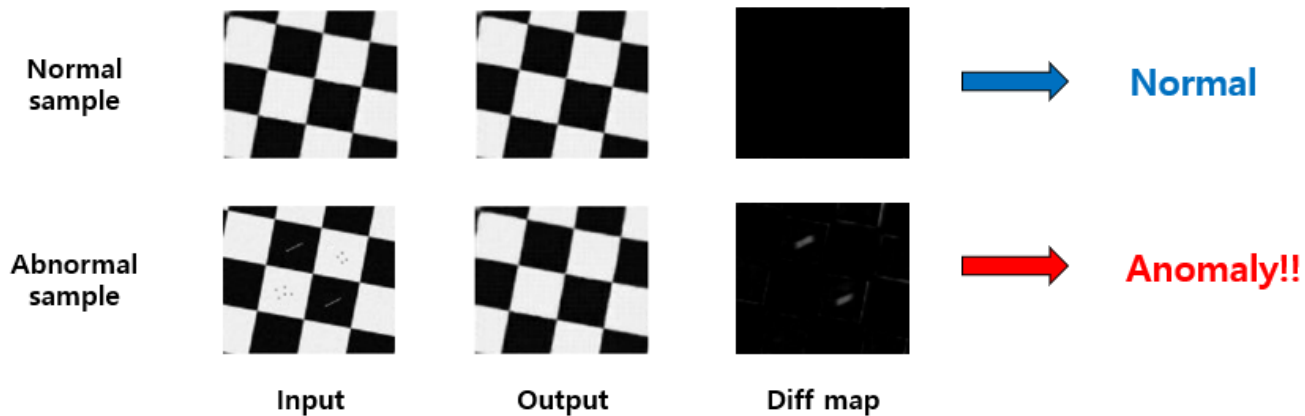
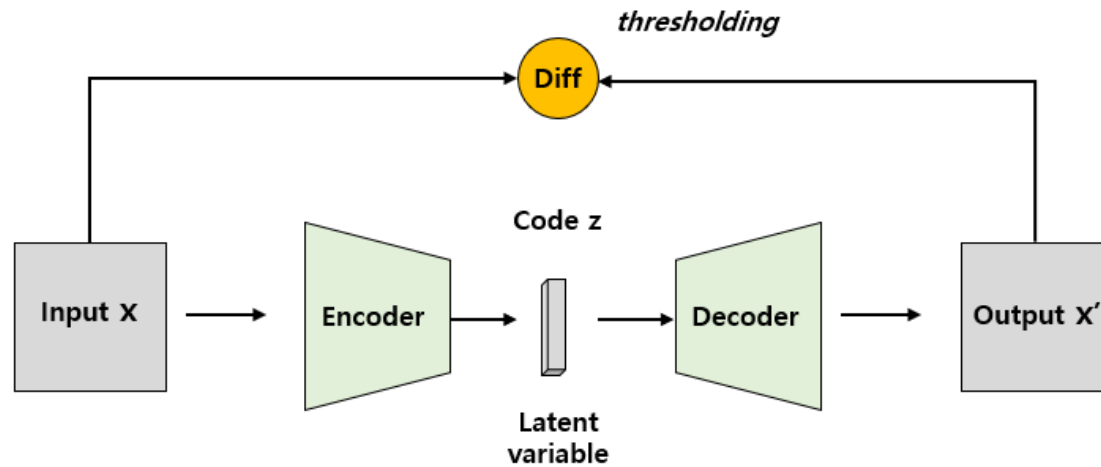
- Autoencoder

: 정상 데이터로 학습된 모델을 사용해 입력 데이터를 재구성하고, 재구성 오차가 큰 데이터를 이상치로 탐지

- 이상 탐지 방식

- 정상 데이터로 Autoencoder 학습
- 새로운 데이터가 들어오면, 이를 모델을 통해 재구성
- 재구성 오차가 큰 데이터를 이상치로 탐지

Autoencoder 이상 탐지



Autoencoder 이상 탐지 in Python

- 데이터 생성

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import mean_squared_error
from keras.models import Model
from keras.layers import Input, Dense
from sklearn.datasets import make_blobs

np.random.seed(42)
X_normal = np.random.randn(300, 2)
X_outliers = np.random.uniform(-5, 5, size=(10, 2))
X = np.vstack([X_normal, X_outliers])
```

Autoencoder 이상 탐지 in Python

■ Autoencoder 모델 정의

```
input_dim = X_scaled.shape[1]
encoding_dim = 2 # 압축 차원

# 인코더
input_layer = Input(shape=(input_dim,))
encoded = Dense(encoding_dim, activation='relu')(input_layer)

# 디코더
decoded = Dense(input_dim, activation='sigmoid')(encoded)

# Autoencoder 모델
autoencoder = Model(input_layer, decoded)

# 인코더 모델 (특성 추출용)
encoder = Model(input_layer, encoded)

# 모델 컴파일
autoencoder.compile(optimizer='adam', loss='mean_squared_error')
```

Autoencoder 이상 탐지 in Python

- 모델 학습 및 재구성 오차 계산

```
autoencoder.fit(X_scaled, X_scaled, epochs=50, batch_size=32,
                shuffle=True, validation_data=(X_scaled, X_scaled), verbose=0)

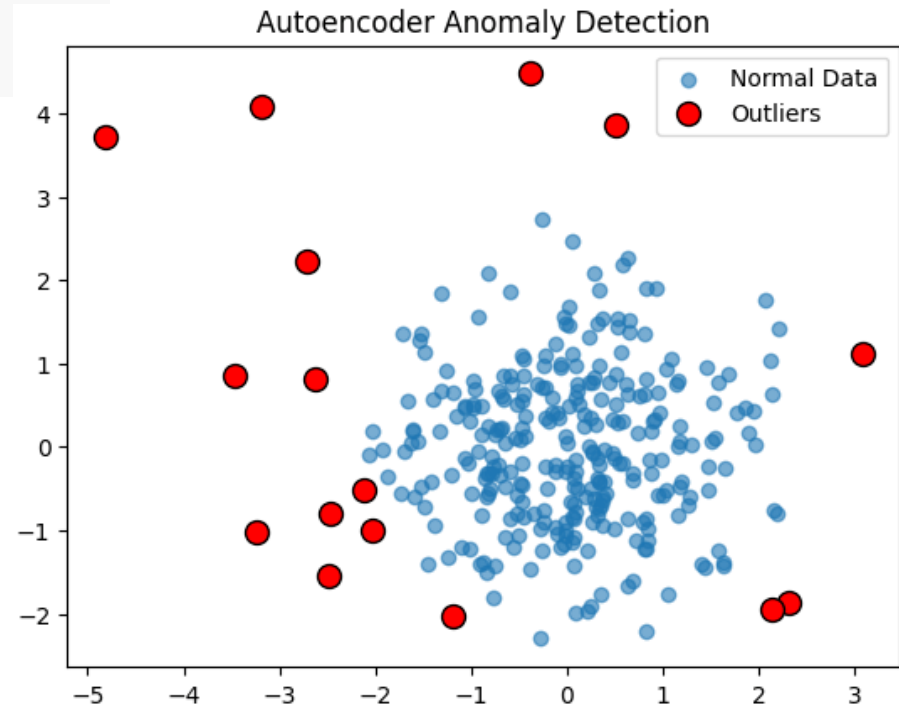
reconstructed = autoencoder.predict(X_scaled)
reconstruction_error = np.mean(np.square(X_scaled - reconstructed), axis=1)

# 이상치 탐지
threshold = np.percentile(reconstruction_error, 95) # 95% 이상을 이상치로 설정
outliers = X[reconstruction_error > threshold]
```

Autoencoder 이상 탐지 in Python

■ 결과 시각화

```
plt.scatter(X[:, 0], X[:, 1], label="Normal Data", alpha=0.6)
plt.scatter(outliers[:, 0], outliers[:, 1], color='red',
            edgecolors='k', s=100, label="Outliers")
plt.legend()
plt.title("Autoencoder Anomaly Detection")
plt.show()
```



Lab 6-2: Unsupervised Outlier Detection

Lab

- NH Data Science Course

- All code examples are written in Python 3
- Please do not hesitate to ask questions!
- TAs
 - 김규태
 - 최예림
- 모든 Lab 자료는 eX-campus에 업로드 되어 있습니다.